VM Host Setup Guide (Linux)

Prerequisites

- Ubuntu 20.04+ or similar Linux distribution
- X11 display server (not Wayland)
- Python 3.10 or higher
- Root/sudo access
- GPU recommended for gaming

System Requirements

Minimum

- 2 CPU cores
- 4GB RAM
- 20GB storage
- 5 Mbps upload bandwidth

Recommended

- 4+ CPU cores
- 8GB+ RAM
- SSD storage
- 10+ Mbps upload bandwidth
- NVIDIA/AMD GPU with hardware encoding

Installation

1. Install System Dependencies

```
sudo apt-get update
sudo apt-get install -y \
  python3 \
  python3-pip \
  python3-venv \
  ffmpeg \
  x11-utils \
  xdotool \
  scrot \
  build-essential
```

2. Install VM Host Service

Automated Installation

```
cd vm-host-linux
sudo ./install.sh
```

The script will:
- Install system dependencies
- Create `/opt/cloud-gaming-vm-host/` directory
- Set up Python virtual environment
- Install Python dependencies
- Create systemd service
- Copy configuration files

Manual Installation

```
# Create installation directory
sudo mkdir -p /opt/cloud-gaming-vm-host
sudo cp -r vm-host-linux/* /opt/cloud-gaming-vm-host/
cd /opt/cloud-gaming-vm-host

# Create virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install --upgrade pip
pip install -r requirements.txt

# Install systemd service
sudo cp vm-host.service /etc/systemd/system/
sudo systemctl daemon-reload
```

3. Configure the Service

Edit `/opt/cloud-gaming-vm-host/.env`:

```
# Signaling Server Configuration
SIGNALING_SERVER_URL=https://signaling.yourdomain.com

# VM Identification
VM_ID=gaming-vm-001
VM_HOSTNAME=gaming-server-01

# Video Configuration
DISPLAY=:0
VIDEO_WIDTH=1920
VIDEO_HEIGHT=1080
VIDEO_FPS=60

# Bitrate Configuration (in kbps)
BITRATE_HIGH=8000
BITRATE_MEDIUM=4000
BITRATE_LOW=2000

# Logging
LOG_LEVEL=INFO
```

Edit `/opt/cloud-gaming-vm-host/config/config.yaml` for advanced settings.

4. Set Up Display

Ensure X11 is running:

```
echo $DISPLAY  # Should show :0 or similar
xdpyinfo       # Should show display information
```

If no display:

```
# Start a virtual display (for headless servers)
sudo apt-get install xvfb
Xvfb :99 -screen 0 1920x1080x24 &
export DISPLAY=:99
```

5. Configure X11 Permissions

```
# Allow access to X11 display
xhost +local:

# Or for specific user
sudo usermod -a -G video $USER
```

6. Start the Service

```
sudo systemctl start vm-host
sudo systemctl enable vm-host
```

7. Verify Installation

```
# Check service status
sudo systemctl status vm-host

# View logs
sudo journalctl -u vm-host -f

# Check for session ID in logs
sudo journalctl -u vm-host | grep "Session ID"
```

You should see output like:

```
VM registered successfully! Session ID: 550e8400-e29b-41d4-a716-446655440000
Clients can connect using session ID: 550e8400-e29b-41d4-a716-446655440000
```

Configuration

Video Quality Presets

Edit `config/config.yaml`:

```yaml
quality_presets:
  high:
    width: 1920
    height: 1080
    fps: 60
    bitrate: 8000
  medium:
    width: 1280
    height: 720
    fps: 60
    bitrate: 4000
  low:
    width: 960
    height: 540
    fps: 60
    bitrate: 2000
  custom:
    width: 2560
    height: 1440
    fps: 144
    bitrate: 15000
```

Network Configuration

Configure STUN/TURN servers in `config/config.yaml`:

```yaml
network:
  stun_servers:
    - "stun:stun.l.google.com:19302"
    - "stun:stun1.l.google.com:19302"
  turn_servers:
    - urls: "turn:turn.yourdomain.com:3478"
      username: "user"
      credential: "password"
```

Input Configuration

```yaml
input:
  enabled: true
  mouse_sensitivity: 1.0
  keyboard_layout: "us"
```

Hardware Encoding

NVIDIA GPU (NVENC)

Install NVIDIA drivers:

```
sudo apt-get install nvidia-driver-535
sudo reboot
```

Verify:

```
nvidia-smi
```

Update FFmpeg command in code to use NVENC:

```
# In screen_capture.py, modify codec to:
codec = "h264_nvenc"
```

AMD GPU (AMF/VCE)

```
sudo apt-get install mesa-va-drivers
```

Update codec:

```
codec = "h264_amf"
```

Intel GPU (Quick Sync)

```
sudo apt-get install intel-media-va-driver
```

Update codec:

```
codec = "h264_qsv"
```

Firewall Configuration

UFW

```
# Allow WebRTC ports (UDP)
sudo ufw allow 10000:20000/udp

# Allow signaling server connection
sudo ufw allow out to any port 3000

sudo ufw enable
```

iptables

```
sudo iptables -A INPUT -p udp --dport 10000:20000 -j ACCEPT
sudo iptables -A OUTPUT -p tcp --dport 3000 -j ACCEPT
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Performance Optimization

1. CPU Governor

Set to performance mode:

```
sudo apt-get install cpufrequtils
sudo cpufreq-set -g performance
```

2. Disable Power Saving

```
# Add to /etc/sysctl.conf
vm.swappiness=10
net.core.rmem_max=134217728
net.core.wmem_max=134217728
```

3. Process Priority

Edit systemd service to add:

```
[Service]
Nice=-10
IOSchedulingClass=realtime
```

4. Dedicated Network Interface

Bind to specific interface in config:

```
network:
  interface: "eth0"
```

Monitoring

System Resources

```
# CPU and memory
htop

# GPU usage (NVIDIA)
watch -n 1 nvidia-smi

# Network bandwidth
iftop
```

Service Logs

```
# Real-time logs
sudo journalctl -u vm-host -f

# Last 100 lines
sudo journalctl -u vm-host -n 100

# Logs since boot
sudo journalctl -u vm-host -b

# Filter by level
sudo journalctl -u vm-host -p err
```

Application Logs

```
tail -f /opt/cloud-gaming-vm-host/vm-host.log
```

Troubleshooting

Service Won’t Start

```
# Check status
sudo systemctl status vm-host

# Check logs
sudo journalctl -u vm-host -n 50

# Test manually
cd /opt/cloud-gaming-vm-host
source venv/bin/activate
python3 -m src.main
```

Can’t Capture Screen

```
# Check display
echo $DISPLAY
xdpyinfo

# Check X11 permissions
xhost +local:

# Test FFmpeg
ffmpeg -f x11grab -video_size 1920x1080 -framerate 60 -i :0 -t 5 test.mp4
```

WebRTC Connection Failed

```
# Check network connectivity
ping signaling.yourdomain.com

# Check firewall
sudo ufw status

# Check STUN server
stunclient stun.l.google.com 19302
```

High CPU Usage

- Enable hardware encoding (NVENC/AMF/QSV)
- Lower video quality preset
- Reduce FPS
- Check for other running processes

Security

1. Run as Non-Root User

Create dedicated user:

```
sudo useradd -r -s /bin/false vmhost
sudo chown -R vmhost:vmhost /opt/cloud-gaming-vm-host
```

Update systemd service:

```
[Service]
User=vmhost
Group=vmhost
```

2. Limit Input Injection

Restrict which keys can be injected:

```
# In input_handler.py
BLOCKED_KEYS = ['power', 'suspend', 'shutdown']
```

3. Network Isolation

Use separate VLAN for gaming VMs.

Scaling

Multiple VMs on One Host

Each VM needs:
- Unique VM_ID
- Separate X display (`:0` , `:1` , `:2` , etc.)
- Dedicated GPU (for best performance)

Auto-Scaling

Integrate with cloud providers:
- AWS EC2 with auto-scaling groups
- Google Cloud Compute Engine
- Azure Virtual Machines

Backup and Recovery

Backup Configuration

```
sudo tar -czf vm-host-config-backup.tar.gz \
  /opt/cloud-gaming-vm-host/.env \
  /opt/cloud-gaming-vm-host/config/
```

Restore

```
sudo tar -xzf vm-host-config-backup.tar.gz -C /
sudo systemctl restart vm-host
```

Next Steps

1. Configure Web Client: SETUP_WEB_CLIENT.md (./SETUP_WEB_CLIENT.md)
2. Test connection from browser
3. Optimize for your specific games

4. Set up monitoring and alerts